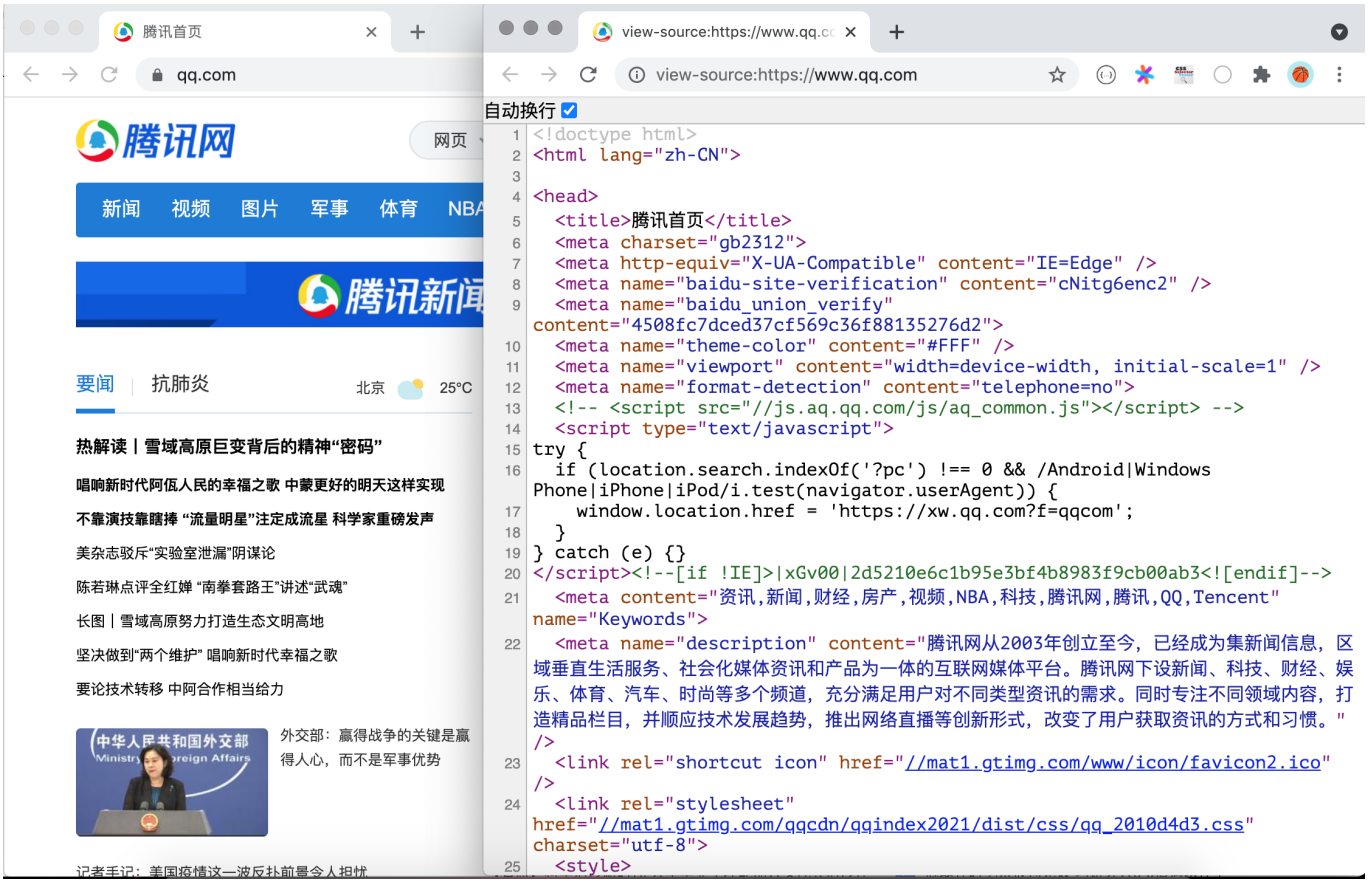
用Python解析HTML页面

在前面的课程中，我们讲到了使用`request`三方库获取网络资源，还介绍了一些前端的基础知识。接下来，我们继续探索如何解析 HTML 代码，从页面中提取出有用的信息。之前，我们尝试过用正则表达式的捕获组操作提取页面内容，但是写出一个正确的正则表达式也是一件让人头疼的事情。为了解决这个问题，我们得先深入的了解一下 HTML 页面的结构，并在此基础上研究另外的解析页面的方法。

HTML 页面的结构

我们在浏览器中打开任意一个网站，然后通过鼠标右键菜单，选择“显示网页源代码”菜单项，就可以看到网页对应的 HTML 代码。



代码的第`1`行是文档类型声明，第`2`行的`<html>`标签是整个页面根标签的开始标签，最后一行是根标签的结束标签`</html>`。`<html>`标签下面有两个子标签`<head>`和`<body>`，放在`<body>`标签下的内容会显示在浏览器窗口中，这部分内容是网页的主体；放在`<head>`标签下的内容不会显示在浏览器窗口中，但是却包含了页面重要的元信息，通常称之为网页的头部。HTML 页面大致的代码结构如下所示。

```
<!doctype html>
<html>
    <head>
        <!-- 页面的元信息，如字符编码、标题、关键字、媒体查询等 -->
    </head>
    <body>
        <!-- 页面的主体，显示在浏览器窗口中的内容 -->
    </body>
</html>
```

标签、层叠样式表（CSS）、JavaScript 是构成 HTML 页面的三要素，其中标签用来承载页面要显示的内容，CSS 负责对页面的渲染，而 JavaScript 用来控制页面的交互式行为。要实现 HTML 页面的解析，可以使用 XPath 的语法，它原本是 XML 的一种查询语法，可以根据 HTML 标签的层次结构提取标签中的内容或标签属性；此外，也可以使用 CSS 选择器来定位页面元素，就跟用 CSS 渲染页面元素是同样的道理。

XPath 解析

XPath 是在 XML（eXtensible Markup Language）文档中查找信息的一种语法，XML 跟 HTML 类似也是一种用标签承载数据的标签语言，不同之处在于 XML 的标签是可扩展的，可以自定义的，而且 XML 对语法有更严格的要求。XPath 使用路径表达式来选取 XML 文档中的节点或者节点集，这里所说的节点包括元素、属性、文本、命名空间、处理指令、注释、根节点等。下面我们通过一个例子来说明如何使用 XPath 对页面进行解析。

```xml
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
    <book>
      <title lang="eng">Harry Potter</title>
      <price>29.99</price>
    </book>
    <book>
      <title lang="zh">Learning XML</title>
      <price>39.95</price>
    </book>
</bookstore>
```

对于上面的 XML 文件，我们可以用如下所示的 XPath 语法获取文档中的节点。

路径表达式	结果
`/bookstore`	选取根元素 bookstore。**注意**：假如路径起始于正斜杠(/)，则此路径始终代表到某元素的绝对路径！
`//book`	选取所有 book 子元素，而不管它们在文档中的位置。
`//@lang`	选取名为 lang 的所有属性。
`/bookstore/book[1]`	选取属于 bookstore 子元素的第一个 book 元素。
`/bookstore/book[last()]`	选取属于 bookstore 子元素的最后一个 book 元素。
`/bookstore/book[last()-1]`	选取属于 bookstore 子元素的倒数第二个 book 元素。
`/bookstore/book[position()<3]`	选取最前面的两个属于 bookstore 元素的子元素的 book 元素。
`//title[@lang]`	选取所有拥有名为 lang 的属性的 title 元素。
`//title[@lang='eng']`	选取所有 title 元素，且这些元素拥有值为 eng 的 lang 属性。
`/bookstore/book[price>35.00]`	选取 bookstore 元素的所有 book 元素，且其中的 price 元素的值须大于 35.00。

路径表达式	结果
`/bookstore/book[price>35.00]/title`	选取 bookstore 元素中的 book 元素的所有 title 元素，且其中的 price 元素的值须大于 35.00。

XPath还支持通配符用法，如下所示。

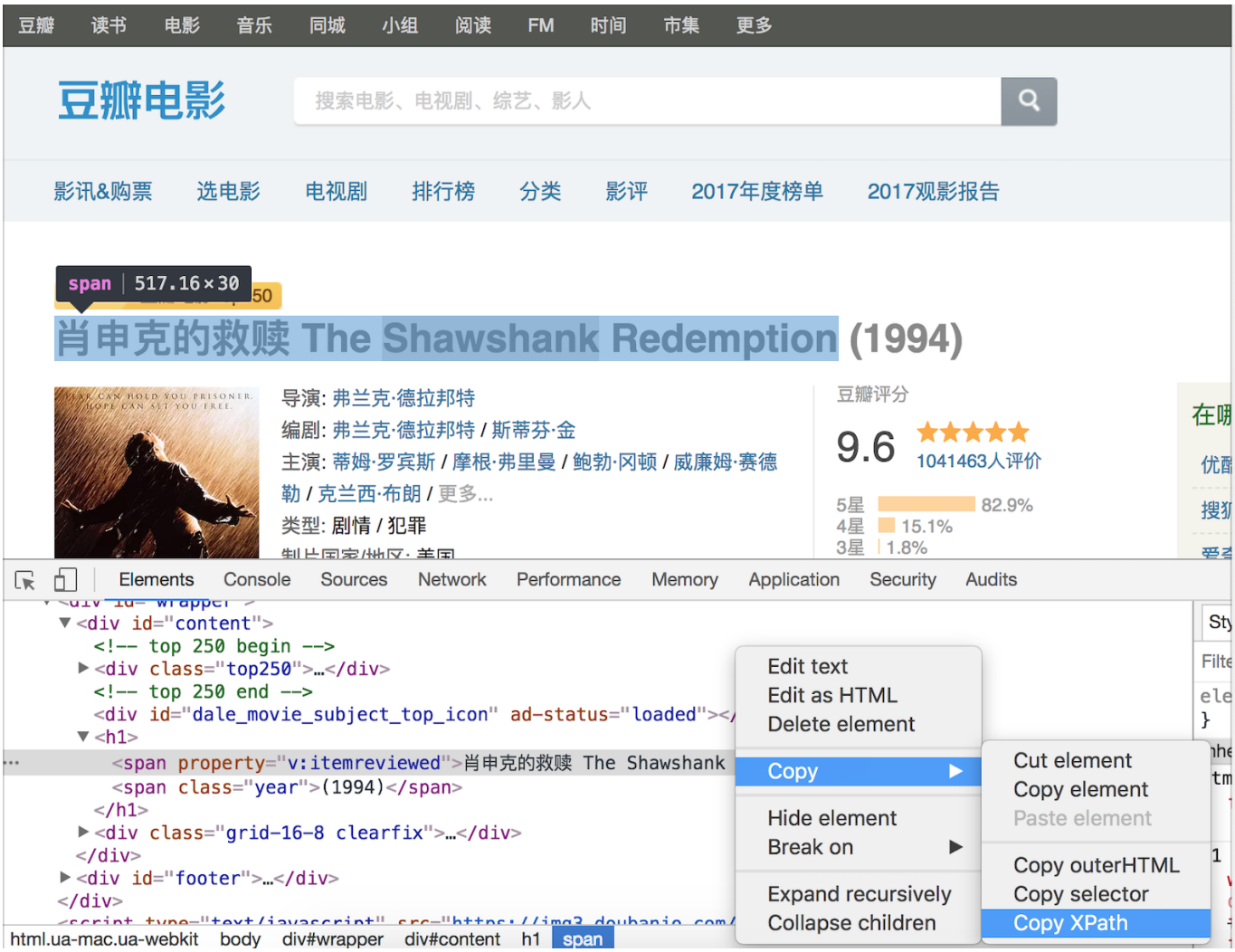
路径表达式	结果
`/bookstore/*`	选取 bookstore 元素的所有子元素。
`//*`	选取文档中的所有元素。
`//title[@*]`	选取所有带有属性的 title 元素。

如果要选取多个节点，可以使用如下所示的方法。

路径表达式	结果
`//book/title | //book/price`	选取 book 元素的所有 title 和 price 元素。
`//title | //price`	选取文档中的所有 title 和 price 元素。
`/bookstore/book/title | //price`	选取属于 bookstore 元素的 book 元素的所有 title 元素，以及文档中所有的 price 元素。

> **说明**：上面的例子来自于“菜鸟教程”网站上的 XPath 教程，有兴趣的读者可以自行阅读原文。

当然，如果不理解或不熟悉 XPath 语法，可以在浏览器的开发者工具中按照如下所示的方法查看元素的 XPath 语法，下图是在 Chrome 浏览器的开发者工具中查看豆瓣网电影详情信息中影片标题的 XPath 语法。



实现 XPath 解析需要三方库`lxml` 的支持，可以使用下面的命令安装`lxml`。

```
pip install lxml
```

下面我们用 XPath 解析方式改写之前获取豆瓣电影 Top250的代码，如下所示。

```
from lxml import etree
import requests

for page in range(1, 11):
    resp = requests.get(
        url=f'https://movie.douban.com/top250?start={(page - 1) * 25}',
        headers={'User-Agent': 'BaiduSpider'}
    )
    tree = etree.HTML(resp.text)
    # 通过XPath语法从页面中提取电影标题
    title_spans = tree.xpath('//*
[@id="content"]/div/div[1]/ol/li/div/div[2]/div[1]/a/span[1]')
    # 通过XPath语法从页面中提取电影评分
    rank_spans = tree.xpath('//*
[@id="content"]/div/div[1]/ol/li[1]/div/div[2]/div[2]/div/span[2]')
```

```
    for title_span, rank_span in zip(title_spans, rank_spans):
        print(title_span.text, rank_span.text)
```

CSS 选择器解析

对于熟悉 CSS 选择器和 JavaScript 的开发者来说，通过 CSS 选择器获取页面元素可能是更为简单的选择，因为浏览器中运行的 JavaScript 本身就可以`document`对象的`querySelector()`和`querySelectorAll()`方法基于 CSS 选择器获取页面元素。在 Python 中，我们可以利用三方库`beautifulsoup4`或`pyquery`来做同样的事情。Beautiful Soup 可以用来解析 HTML 和 XML 文档，修复含有未闭合标签等错误的文档，通过为待解析的页面在内存中创建一棵树结构，实现对从页面中提取数据操作的封装。可以用下面的命令来安装 Beautiful Soup。

```
pip install beautifulsoup4
```

下面是使用`bs4`改写的获取豆瓣电影Top250电影名称的代码。

```
import bs4
import requests

for page in range(1, 11):
    resp = requests.get(
        url=f'https://movie.douban.com/top250?start={(page - 1) * 25}',
        headers={'User-Agent': 'BaiduSpider'}
    )
    # 创建BeautifulSoup对象
    soup = bs4.BeautifulSoup(resp.text, 'lxml')
    # 通过CSS选择器从页面中提取包含电影标题的span标签
    title_spans = soup.select('div.info > div.hd > a > span:nth-child(1)')
    # 通过CSS选择器从页面中提取包含电影评分的span标签
    rank_spans = soup.select('div.info > div.bd > div > span.rating_num')
    for title_span, rank_span in zip(title_spans, rank_spans):
        print(title_span.text, rank_span.text)
```

关于 BeautifulSoup 更多的知识，可以参考它的官方文档。

总结

下面我们对三种解析方式做一个简单比较。

解析方式	对应的模块	速度	使用难度
正则表达式解析	`re`	快	困难
XPath 解析	`lxml`	快	一般
CSS 选择器解析	`bs4`或`pyquery`	不确定	简单